Chess Outcome Prediction

Big Data Pipeline from Move-Level Board States

Ivan Chabanov, Alexander Mikhailov, Danil Eramasov, Bulat Fakhrutdinov

Big Data Course -- IU, 2026

1. Project Goal

We build a scalable Big Data system for chess analytics.

Main task:

```
board state + game context at a selected ply
        -> white_win | black_win | draw
```

The project covers the full path from raw Chess.com archives to distributed storage, EDA, dashboard, and Spark ML evaluation.

2. Dataset

Source: Chess.com public archive API.

Unit of observation: **one ply**: one half-move from one game.

Property	Value
Rows	674,594 plies
Size	589.22 MB
Features	100+
Target	`final_result_class`
Classes	`white_win`, `black_win`, `draw`

The dataset satisfies the course scale requirements.

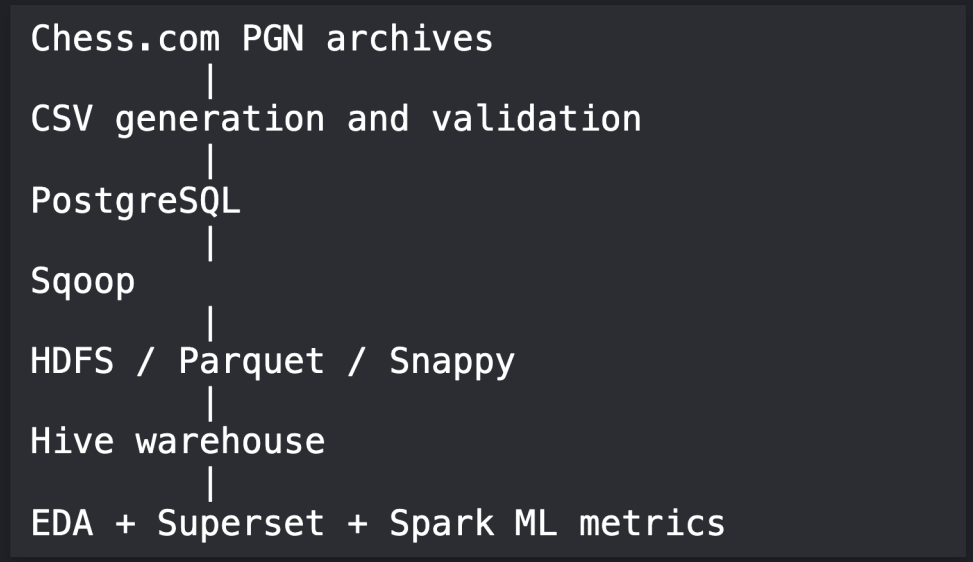
3. Feature Space

The dataset combines game context and chess-specific board features.

Group	Examples
Game metadata	players, ratings, time control, ECO opening
Move context	SAN/UCI move, side to move, ply index
Board state	FEN, material, legal move count, phase
Derived features	pawn structure, king safety, castling, time pressure
Target	final game outcome

Leakage columns such as raw result and termination are kept for audit only.

4. Pipeline Architecture



This is the core Big Data part of the project.

5. Storage Design

The main analytical table is `chess_moves`.

Layer	Decision
PostgreSQL	schema and initial ingestion
HDFS	distributed storage
Hive	external warehouse table
Format	Parquet
Compression	Snappy
Partitioning	`archive_month`
Bucketing	`game_uuid`, 8 buckets

The denormalized ply-level table is easier for Spark and Hive scans.

6. EDA and Dashboard

Stage II produces eight HiveQL insights.

Query	Question
q1	outcome by time control
q2	top ECO openings by white win rate
q3	rating gap vs outcome
q4	game length by time control
q5	checkmate phase distribution
q6	outcome trends over time
q7	outcome by termination type
q8	material advantage vs outcome

CSV outputs feed Superset charts and dashboard interpretation. Game-level insights use `ply_index = 1` to avoid over-weighting long games.

7. ML Results

> The ML metrics are now available in the pipeline artifacts: `output/model_metrics.csv` and `output/phase_metrics.csv`.

Task: multiclass classification of `final_result_class`.

Model	Accuracy	Weighted F1
Naive Bayes	0.679	0.654
Logistic Regression	0.684	0.660
Random Forest	**0.706**	**0.681**

Random Forest is the strongest overall model.

8. ML by Game Phase

Best model by phase:

Phase	Best model	Samples	Accuracy	Weighted F1
Opening	Random Forest	50,968	0.688	0.674
Middlegame	Random Forest	92,383	0.715	0.678
Endgame	Random Forest	48,051	**0.762**	**0.713**

The model performs best in the endgame, where board-state features carry stronger outcome signal.

9. Final Takeaway

What is ready now:

1. Chess.com data is transformed into a 674k-row ply-level dataset.
2. The project implements PostgreSQL -> Sqoop -> HDFS -> Hive.
3. Hive storage uses Parquet, Snappy, partitioning, and bucketing.
4. Eight EDA queries support the Superset dashboard.
5. Spark ML metrics are available; Random Forest reaches 0.706 accuracy and 0.681 weighted F1 overall.

This version is short enough for a defense talk and now includes the ML results.